



UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO

DISCIPLINA: LABORATÓRIO DE ALGORITMO E ESTRUTURAS DE DADOS II

CURSO: BACHARELADO INTERDISCIPLINAR EM TECNOLOGIA DA
INFORMAÇÃO

DOCENTE: KENNEDY REURISON LOPES

DISCENTE(S): EDUARDO FERNANDES PINHEIRO, JOSÉ LUÍS FARIAS BESSA,
JOÃO PEDRO NOGUEIRA FERNANDES, HUDSON DIEGO DE OLIVEIRA SENA

SISTEMA DE CONSULTA EFICIENTE COM TABELA HASH E FILTRO DE BLOOM

Sumário

1	Objetivo	2
2	Instruções de Compilação e Execução	2
3	Implementação e Decisões de Projeto	2
3.1	Tabela Hash	2
3.2	Filtro de Bloom	3
4	Experimentos	3
5	Análise	3
6	Conclusão	4

Objetivo

Este trabalho implementa um sistema de verificação de cadastro de usuários utilizando duas estruturas complementares: **Tabela Hash** (armazenamento e recuperação exata, com encadeamento externo) e **Filtro de Bloom** (estrutura probabilística que filtra consultas negativas antes de acessar a hash, reduzindo acessos desnecessários). O objetivo é analisar o impacto combinado dessas estruturas em desempenho, memória e taxa de falsos positivos.

Instruções de Compilação e Execução

Requisitos: GCC 9 ou superior, libm, Linux.

Obter o código:

```
# via SSH
git clone git@github.com:JoaoPedroNFernandes/projeto-Ed2.git

# via HTTPS
git clone https://github.com/JoaoPedroNFernandes/projeto-Ed2.git

cd projeto-Ed2
```

Compilar e gerar dados de teste:

```
make          # compila -> src/main
make gerar    # gera usuarios1000.txt, usuarios10000.txt, usuarios100000.txt
make clean    # remove binarios e dados gerados
```

Executar:

```
./src/main
```

O programa exibe um menu com as opções: inserir usuário, consultar com/sem Bloom Filter, inserir em lote a partir de arquivo, exibir estatísticas e executar experimento. Identificadores seguem o formato [8 letras][3 dígitos], ex.: islaifda122.

Implementação e Decisões de Projeto

Tabela Hash

Implementada com **encadeamento externo** e função hash **djb2** ($\text{hash} = \text{hash} * 33 + c$), conhecida pela boa distribuição em strings curtas.

Tamanho escolhido: $m = 131071 = 2^{17} - 1$ (primo de Mersenne). O uso de primo reduz agrupamentos (*clustering*). Para $n = 100.000$ elementos, o fator de carga resultante é:

$$\alpha = \frac{n}{m} = \frac{100.000}{131.071} \approx 0,76$$

Para encadeamento externo, $\alpha < 1,0$ garante custo médio de busca $O(1 + \alpha)$ — praticamente constante. O mesmo tamanho de tabela foi usado para todos os cenários, o que explica os fatores de carga muito diferentes entre 1.000 e 100.000 elementos, mas ainda assim com desempenho adequado (ver Tabela 2).

Filtro de Bloom

Implementado com **empacotamento real de bits** (1 bit por posição, 8 por byte) e **double hashing** (Kirsch & Mitzenmacher [2]), que simula k funções independentes a partir de duas bases (djb2 e FNV-1a):

$$g_i(x) = (h_1(x) + i \cdot h_2(x)) \bmod m, \quad i = 0, \dots, k-1$$

Dimensionamento automático via `bloom_create(n, p)`, usando as fórmulas clássicas [1]:

$$m = \left\lceil \frac{-n \ln p}{(\ln 2)^2} \right\rceil \quad (1)$$

$$k = \left\lceil \frac{m}{n} \ln 2 \right\rceil \quad (2)$$

Para $n = 100.000$ e $p = 1\%$: $m = 958.506$ bits (≈ 117 KB) e $k = 7$.

Experimentos

Cada cenário carrega n usuários do arquivo, depois mede o tempo de busca de todos os elementos (i) direto na hash e (ii) com Bloom antes da hash. A taxa de falsos positivos é medida com n consultas de nomes aleatórios não inseridos.

Tabela 1: Resultados dos experimentos

Quantidade	T. sem Bloom (s)	T. com Bloom (s)	Falsos Positivos
1.000	0,000170	0,000216	2,80%
10.000	0,002302	0,002780	0,95%
100.000	0,030613	0,038802	2,67%

Tabela 2: Dimensionamento do Bloom e da hash por cenário

n	m (bits)	k	Memória Bloom	α (hash)	Colisões
1.000	9.586	7	1,17 KB	0,0076	6
10.000	95.851	7	11,70 KB	0,0763	371
100.000	958.506	7	117,01 KB	0,7629	29.948

Análise

1. O Filtro de Bloom reduziu consultas na Tabela Hash? Nos testes, consultamos apenas elementos presentes — o Bloom sempre respondeu “possivelmente existe” e a hash foi acessada de qualquer forma. O benefício real ocorre em **consultas negativas**: o Bloom elimina matematicamente falsos negativos e evita o acesso à hash por completo. Em produção, com alta frequência de tentativas de identificadores inválidos, a redução de acessos seria expressiva.

2. Como o tamanho do vetor impactou os resultados? m cresce linearmente com n para uma taxa p fixada (equação 1). Nos três cenários, o vetor foi calibrado para $p = 1\%$, mantendo a taxa teórica estável em $\approx 1\%$. A variação observada (0,95%–2,80%) é esperada pela natureza estocástica da amostragem. Um vetor subdimensionado aumentaria os FP; superdimensionado desperdiçaria memória sem ganho proporcional.

3. O número de funções hash alterou a precisão? Sim. O valor $k = 7$ (obtido pela equação 2) é o ponto ótimo que minimiza os FP para o m calculado. Com k menor, cada elemento marca poucos bits e colisões acidentais são mais prováveis; com k maior, a taxa de ocupação do vetor cresce rapidamente, aumentando também os FP — efeito contrário ao desejado.

4. Em qual cenário o Bloom foi mais vantajoso? No cenário de **100.000 registros**: a hash apresentou fator de carga $\alpha = 0,76$ com 29.948 colisões (bucket máximo de 7 elementos), tornando cada busca mais custosa. O vetor do Bloom (117 KB) cabe inteiro no cache L2/L3 do processador, mantendo o custo de consulta baixo. Com consultas negativas, cada acesso à hash evitado representaria a maior economia relativa.

Conclusão

A combinação de Tabela Hash com Filtro de Bloom mostrou-se eficaz para sistemas com grandes volumes de dados. O Bloom elimina consultas negativas sem falsos negativos, reduzindo o acesso à estrutura principal. O dimensionamento correto — derivado pelas fórmulas de m e k — é essencial: o filtro atingiu a taxa de $\approx 1\%$ de FP conforme projetado em todos os cenários, com memória proporcional a n e custo de consulta constante.

Referências

- [1] BLOOM, B. H. **Space/time trade-offs in hash coding with allowable errors.** *Communications of the ACM*, v. 13, n. 7, p. 422–426, 1970.
- [2] KIRSCH, A.; MITZENMACHER, M. **Less hashing, same performance: Building a better Bloom filter.** *Random Structures & Algorithms*, v. 33, n. 2, p. 187–218, 2008.
- [3] CORMEN, T. H. et al. **Introduction to Algorithms.** 4. ed. MIT Press, 2022.
- [4] KNUTH, D. E. **The Art of Computer Programming**, v. 3: Sorting and Searching. 2. ed. Addison-Wesley, 1998.